



Chapter 10

JavaScript: Arrays

Internet & World Wide Web
How to Program, 5/e

<ftp://111webdesign:webdesign@120.114.190.22:4000>





OBJECTIVES

In this chapter you'll:

- Declare arrays, initialize arrays and refer to individual elements of arrays.
- Store lists and tables of values in arrays.
- Pass arrays to functions.
- Search and sort arrays.
- Declare and manipulate multidimensional arrays.



10.1 Introduction

▶ Arrays 陣列

- Data structures consisting of related data items

- 註：儲存同類資料的結構

▶ JavaScript arrays

- Can “**dynamically**” (動態) **change size** (變更大小) after they are created

▶ An array is a **group of memory locations**

- All have the **same name** (相同變數名稱) and normally are of the **same type** (相同型態)

- 註：陣列是一群記憶體位置，有相同 變數名稱、相同 型態
 - 註：陣列內個別的記憶體位置，稱為“**element** (元素)”
 - 註：當要存取其中的element時，使用陣列名稱**[編號]**，其中**中括號**是描述第幾個位置的element。



10.2 Arrays (Cont.)

arrayname[元素編號] //陣列

- ▶ The first element in every array is the zeroth element.
 - 註：第一個element是編號0, ex: **A[0]**
- ▶ The **i-th element** of array **c** is referred to as **c[i-1]**.
 - 註：第 **i**個元素的編號為**i-1**.

(array是一種物件)

(∵長度可能會變動)

arrayname.length //陣列長度

- ▶ In JavaScript, every array knows its own **length** (長度可知), which it stores in its **length attribute** (長度屬性) and can be found

(元素編號)		(陣列名稱)
Position number of the element within the array c		Name of the array is c
c[0]	-45	(存放內容) Value of array element c[4]
c[1]	6	
c[2]	0	
c[3]	72	
c[4]	1543	
c[5]	-89	
c[6]	0	
c[7]	62	
c[8]	-3	
c[9]	1	
c[10]	6453	
c[11]	78	

10.2 Arrays (Cont.)

Operators	Associativity	Type
() [] . [...]陣列的中括號關聯性優先權最高	left to right	highest //最高優先權(關聯性)
++ -- !	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment

Fig. 10.2 | Precedence and associativity of the operators discussed so far.

10.3 Declaring and Allocating Arrays



- ▶ JavaScript arrays are **Array objects** (陣列物件).

(新建)
var 陣列名稱 = **new Array**(元素個數); //陣列宣告

- 範例: var x = new Array(5); //長度5的陣列x
- ▶ You use the **new operator** (新建) to create an array and to specify the **number of elements** (元素個數) in an array.
- ▶ The **new** operator **creates an object** (創建物件) as the script executes by **obtaining enough memory** (預留記憶體) to store an object of the type specified to the right of new.
 - 註: **new**是用來創建一個物件，使物件獲得充份的記憶體。
- ▶ JavaScript **reallocates** (重新配置) an Array when a value is assigned to an element that is **outside** the bounds of the original Array
 - 註: 若寫入的元素編號超出原本宣告長度，JavaScript會重新配置此陣列元素數 (例如: 宣告長度10元素，但寫入第11個元素)

10.3 Declaring and Allocating Arrays

▶ Example:

- 指定與未指定陣列長度

```
1) InitArray.html x InitArray.js x
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.3: InitArray.html -->
4 <!-- Web page for showing the results of initializing arrays. -->
5
6 <html>
7   <head>
8     <meta charset = "utf-8">
9     <title>Initializing an Array</title>
10    <link rel = "stylesheet" type = "text/css" href = "tablestyle.css">
11    <script src = "InitArray.js"></script> (JavaScript來源)
12  </head>
13  <body>
14    <div id = "output1"></div> (2個空id, 輸出用)
15    <div id = "output2"></div>
16  </body>
17 </html>
```

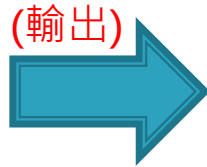
```
1) InitArray.html x InitArray.js x
(2) start函式: function start()
4 {
5   var n1 = new Array( 5 ); // allocate five-element array (陣列1: 長度5)
6   var n2 = new Array(); // allocate empty array (陣列2: 未指定長度, 待寫入時決定)
7
8   // assign values to each element of array n1
9   var length = n1.length; // get array's length once before the loop
10
11   for ( var i = 0; i < n1.length; ++i )
12   {
13     n1[ i ] = i; (取得"長度屬性")
14   } // end for
15
16   // create and initialize five elements in array n2
17   for ( i = 0; i < 5; ++i ) (指定"長度=5")
18   {
19     n2[ i ] = i;
20   } // end for
21
22   outputArray( "Array n1:", n1, document.getElementById( "output1" ) ); (函式)
23   outputArray( "Array n2:", n2, document.getElementById( "output2" ) );
24 } // end function start
25
26 // output the heading followed by a two-column table
27 // containing indices and elements of "theArray"
28 function outputArray( heading, theArray, output ) (文字說明)(陣列變數)(輸出id物件)
29 {
30   var content = "<h2>" + heading + "</h2><table>" +
31     "<thead><th>Index</th><th>Value</th></thead><tbody>";
32
33   // output the index and value of each array element
34   var length = theArray.length; // get array's length once before loop (寫入列數=陣列長度)
35
36   for ( var i = 0; i < length; ++i )
37   {
38     content += "<tr><td>" + i + "</td><td>" + theArray[ i ] +
39       "</td></tr>";
40   } // end for
41
42   content += "</tbody></table>"; (輸出html語法)
43   output.innerHTML = content; // place the table in the output element
44 } // end function outputArray
45
46 window.addEventListener( "load", start, false );
47
```

(1) 註冊event handler: 當load時, 執行start函式

10.3 Declaring and Allocating Arrays



► Output:



(陣列寫入時才決定長度・跟c語言不同)

Index	Value
0	0
1	1
2	2
3	3
4	4

Index	Value
0	0
1	1
2	2
3	3
4	4

(陣列n1: 長度5)

(陣列n2: 長度也為5)



10.4 Examples Using Arrays (Cont.)

陣列宣告 & 初始化 (範例一)

var 陣列1 = [2, 4, 6, 8]; //使用中括弧，很特別

var 陣列2 = [2, , , 8]; //還未知要放什麼值

► Arrays can be created using a comma-separated **initializer list** (初始序列) enclosed in **square brackets** ([...])

- 註：中括號內放入初始元素，利用“逗號”分開
- The **array's size** (陣列長度) is determined by the **number of values** in the initializer list
- 註：陣列長度取決於幾筆數值放入初始序列。

陣列宣告 & 初始化 (範例二)

var 陣列名稱 = new Array("...", "...", "...", ..); //new + array小括弧

► The initial values of an array can be specified as **arguments** in the **parentheses** (小括弧) following **new** Array

- The **size of the array** (陣列長度) is determined by the number of values in parentheses
- 註：陣列長度也取決於幾筆數值放入初始序列。

10.4.2 Initializing Arrays with Initializer Lists

► Example:

■ 不同陣列宣告&初始化

```
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.5: InitArray2.html -->
4 <!-- Web page for showing the results of initializing arrays. -->
5
6 <html>
7   <head>
8     <meta charset = "utf-8">
9     <title>Initializing an Array</title>
10    <link rel = "stylesheet" type = "text/css" href = "tablestyle.css">
11    <script src = "InitArray2.js"></script>
12  </head>
13  <body>
14    <div id = "output1"></div>
15    <div id = "output2"></div>
16    <div id = "output3"></div>
17  </body>
18 </html>
```

(JavaScript來源)

(3個空id · 輸出用)

```
1 // Fig. 10.6: InitArray2.js
2 // Initializing arrays with initializer lists.
3
4 function start()
5 {
6   // Initializer list specifies the number of elements and
7   // a value for each element.
8   var colors = new Array( "cyan", "magenta", "yellow", "black" );
9   var integers1 = [ 2, 4, 6, 8 ];
10  var integers2 = [ 2, , , 8 ];
11
12  outputArray( "Array colors contains", colors,
13    document.getElementById( "output1" ) );
14  outputArray( "Array integers1 contains", integers1,
15    document.getElementById( "output2" ) );
16  outputArray( "Array integers2 contains", integers2,
17    document.getElementById( "output3" ) );
18 } // end function start
19
20 // output the heading followed by a two-column table
21 // containing indices and elements of "theArray"
22 function outputArray( heading, theArray, output )
23 {
24   var content = "<h2>" + heading + "</h2><table>" +
25     "<thead><th>Index</th><th>Value</th></thead><tbody>";
26
27   // output the index and value of each array element
28   var length = theArray.length; // get array's length once before loop
29
30   for ( var i = 0; i < length; ++i )
31   {
32     content += "<tr><td>" + i + "</td><td>" + theArray[ i ] +
33       "</td></tr>";
34   } // end for
35
36   content += "</tbody></table>";
37   output.innerHTML = content; // place the table in the output element
38 } // end function outputArray
39
40 window.addEventListener( "load", start, false );
```

(陣列1: 長度4;

陣列2: 長度4;

陣列3: 長度4 · 剩餘二元素內容未指定)

(輸出陣列)

(3) 輸出陣列

(文字說明)(陣列變數)(id物件 · 輸出用)

(寫入列數=陣列長度)

(1) 註冊event handler: 當load時 · 執行start函式

10.4.2 Initializing Arrays with Initializer Lists



(輸出)



Array colors contains	
Index	Value
0	cyan
1	magenta
2	yellow
3	black

Array integers1 contains	
Index	Value
0	2
1	4
2	6
3	8

Array integers2 contains	
Index	Value
0	2
1	undefined
2	undefined
3	8

(陣列1: 長度4 · 4項初始值 : cyan, magenta, yellow, black)

(陣列2: 長度4 · 4項初始值 : 2, 4, 6, 8)

(陣列3: 長度4 · 2項初始值、2項未定義)

10.4.3 Summing the Elements of an Array with **for** and **for...in**

(自動計算陣列元素個數，較精簡)

- ▶ Example: JavaScript's **for...in** Repetition Statement
 - Enables a script to perform a task for **each element in an array**

The image shows a web browser window and a code editor. The browser window displays the output of the script: "Total using indices: 55" and "Total using for...in: 55". The code editor shows the HTML and JavaScript code. The HTML code includes a script tag for SumArray.js and a div with id="output". The JavaScript code defines an array, calculates the sum using both for and for...in loops, and updates the output div. Annotations in Chinese explain the code: (1) 註冊event handler: 當load時，執行start函式; (2) start函式; (陣列宣告); (JavaScript來源); (1個輸出id); (輸出); (利用for 迴圈); (利用for...in迴圈:累加).

```
3) SumArray.html
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.7: SumArray.html -->
4 <!-- HTML5 document that displays the sum of an array -->
5 <html>
6 <head>
7   <meta charset = "utf-8">
8   <title>Sum Array Elements</title>
9   <script src = "SumArray.js"></script>
10 </head>
11 <body>
12   <div id = "output"></div>
13 </body>
14 </html>
```

(JavaScript來源)

(1個輸出id)

(輸出)

```
3) SumArray.html SumArray.js
1 // Fig. 10.8: SumArray.js
2 // Summing the elements of an array with for and for...in
3 function start()
4 {
5   var theArray = [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ];
6
7   var total1 = 0, total2 = 0;
8
9   // iterates through the elements of the array in order and adds
10  // each element's value to total1
11  var length = theArray.length; // get array's length once before loop
12
13  for ( var i = 0; i < theArray.length; ++i )
14  {
15    total1 += theArray[ i ];
16  } // end for
17
18  var results = "<p>Total using indices: " + total1 + "</p>";
19
20  // iterates through the elements of the array using a for... in
21  // statement to add each element's value to total2
22  for ( var element in theArray )
23  {
24    total2 += theArray[ element ];
25  } // end for
26
27  results += "<p>Total using for...in: " + total2 + "</p>";
28  document.getElementById( "output" ).innerHTML = results;
29 } // end function start
30
31 window.addEventListener( "load", start, false );
32
```

(2) start函式

(陣列宣告)

(利用for 迴圈)

(利用for...in迴圈:累加)

(1) 註冊event handler: 當load時，執行start函式

Total using indices: 55 (利用for 迴圈)

Total using for...in: 55 (利用for...in迴圈)

Question #1

- ▶ 思考: What is the **output** (輸出結果) if we change **Line #5** in previous example (p.13) to the following code?

- `var theArray = [ 1, 2, , , 5, 6, 7, 8, 9, 10 ];`

(若未定義)

試試看!

10.4.4 Using the Elements of an Array as Counters

- Example: (程式改寫：陣列版本) To roll 12 dice at a time and kept statistics (統計) showing the number of times and the percentage (頻率) of the time each face occurred.

```
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.9: RollDice.html -->
4 <!-- HTML5 document for the dice rolling example. -->
5 <html>
6 <head>
7 <meta charset = "utf-8">
8 <title>Die Rolling Frequencies</title>
9 <link rel = "stylesheet" type = "text/css" href = "style.css">
10 <script src = "RollDice.js"></script>
11 </head>
12 <body>
13 <p><img id = "die1" src = "blank.png" alt = "die 1 image">
14 <img id = "die2" src = "blank.png" alt = "die 2 image">
15 <img id = "die3" src = "blank.png" alt = "die 3 image">
16 <img id = "die4" src = "blank.png" alt = "die 4 image">
17 <img id = "die5" src = "blank.png" alt = "die 5 image">
18 <img id = "die6" src = "blank.png" alt = "die 6 image"></p>
19 <p><img id = "die7" src = "blank.png" alt = "die 7 image">
20 <img id = "die8" src = "blank.png" alt = "die 8 image">
21 <img id = "die9" src = "blank.png" alt = "die 9 image">
22 <img id = "die10" src = "blank.png" alt = "die 10 image">
23 <img id = "die11" src = "blank.png" alt = "die 11 image">
24 <img id = "die12" src = "blank.png" alt = "die 12 image"></p>
25 <form action = "#">
26 <input id = "rollButton" type = "button" value = "Roll Dice">
27 </form>
28 <div id = "frequencyTableDiv"></div>
29 </body>
30 </html>
```

```
1 // Fig. 10.10: RollDice.js
2 // Summarizing die rolling frequencies with an array instead of switch
3 var frequency = [ , 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ]; // frequency[0] uninitialized
4 var totalDice = 0;
5 var dieImages = new Array(12); // array to store img elements
6
7 // get die img elements
8 function start()
9 {
10     var button = document.getElementById( "rollButton" );
11     button.addEventListener( "click", rollDice, false );
12     var length = dieImages.length; // get array's length once before loop
13
14     for ( var i = 0; i < length; ++i )
15     {
16         dieImages[ i ] = document.getElementById( "die" + ( i + 1 ) );
17     } // end for
18 } // end function start
19
20 // roll the dice
21 function rollDice()
22 {
23     var face; // face rolled
24     var length = dieImages.length;
25
26     for ( var i = 0; i < length; ++i )
27     {
28         face = Math.floor( 1 + Math.random() * 6 ); // (產生隨機點數)
29         tallyRolls( face ); // increment a frequency counter
30         setImage( i, face ); // display appropriate die image
31         ++totalDice; // increment total
32     } // end for
33
34     updateFrequencyTable();
35 } // end function rollDice
```


10.4.4 Using the Elements of an Array as Counters



(4) 計算次數函式

```
38 // increment appropriate frequency counter
39 function tallyRolls( face )
40 {
41     ++frequency[ face ]; // increment appropriate counter
42 } // end function tallyRolls
```

(5) 變更圖片函式

```
43 // set image source for a die
44 function setImage( dieNumber, face )
45 {
46     dieImages[ dieNumber ].setAttribute( "src", "die" + face + ".png" );
47     dieImages[ dieNumber ].setAttribute( "alt",
48         "die with " + face + " spot(s)" );
49 } // end function setImage
```

(6) 產生Table函式

```
50 // update frequency table in the page
51 function updateFrequencyTable()
52 {
53     {
54         var results = "<table><caption>Die Rolling Frequencies</caption>" +
55             "<thead><th>Face</th><th>Frequency</th>" +
56             "<th>Percent</th></thead><tbody>";
57         var length = frequency.length;
58
59         // create table rows for frequencies
60         for ( var i = 1; i < length; ++i )
61         {
62             results += "<tr><td>i</td><td>" + frequency[ i ] + "</td><td>" +
63                 formatPercent(frequency[ i ] / totalDice) + "</td></tr>";
64         } // end for
65
66         results += "</tbody></table>";
67         document.getElementById( "frequencyTableDiv" ).innerHTML = results;
68     } // end function updateFrequencyTable
69
70     // format percentage
71     function formatPercent( value )
72     {
73         value *= 100;
74         return value.toFixed(2);
75     } // end function formatPercent
76
77     window.addEventListener( "load", start, false );
```

(利用array指定元素++; 先前是用switch 6種case統計)

(利用for迴圈產生Table行列; 先前是寫6次face統計)

Die Rolling Frequencies

Roll Dice

Face	Frequency	Percent
1	38	14.39
2	48	18.18
3	38	14.39
4	40	15.15
5	36	13.64
6	64	24.24

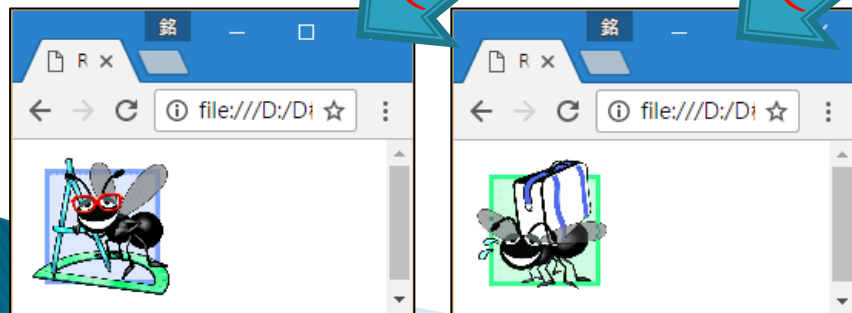
(1) 註冊event handler: 當load時·執行start函式

10.5 Random Image Generator Using Arrays

- ▶ Example: (採用陣列方式) the random image generator required image files to be named.
- ▶ 範例連結

```
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.11: RandomPicture.html -->
4 <!-- HTML5 document that displays randomly selected images. -->
5 <html>
6 <head>
7   <meta charset = "utf-8">
8   <title>Random Image Generator</title>
9   <script src = "RandomPicture.js"></script> (JavaScript來源)
10 </head>
11 <body>
12   <img id = "image" src = "CPE.png" alt = "Common Programming Error"> (1個空id · 輸出用)
13 </body>
14 </html>
```

```
1 // Fig. 10.12: RandomPicture2.js
2 // Random image selection using arrays (陣列宣告: 7種圖片檔名)
3 var iconImg;
4 var pictures = [ "CPE", "EPT", "GPP", "GUI", "PERF", "PORT", "SEO" ];
5 var descriptions = [ "Common Programming Error",
6   "Error-Prevention Tip", "Good Programming Practice",
7   "Look-and-Feel Observation", "Performance Tip", "Portability Tip",
8   "Software Engineering Observation" ];
9
10 // pick a random image and corresponding description then modify
11 // the img element in the document's body
12 function pickImage() (3) 挑圖片函式:
13 { (隨機產生 0 ~ 6 數字 · 作為index)
14   var index = Math.floor( Math.random() * 7 );
15   iconImg.setAttribute( "src", pictures[ index ] + ".png" );
16   iconImg.setAttribute( "alt", descriptions[ index ] ); (更換圖片名稱)
17 } // end function pickImage
18
19 // registers iconImg's click event handler
20 function start() (2) start函式
21 { (3) 註冊event handler: 當圖片click時 · 執行pickImage函式
22   iconImg = document.getElementById( "image" );
23   iconImg.addEventListener( "click", pickImage, false );
24 } // end function start
25
26 window.addEventListener( "load", start, false );
27
```



(1) 註冊event handler: 當load時 · 執行start函式



10.6 References and Reference Parameters

兩種傳遞參數至函式(或方法)

- ▶ **Two ways** to **pass arguments** to **functions** (or **methods**)
 - **pass-by-value** (傳值)
 - **pass-by-reference** (傳參考)
- ▶ **(1) Pass-by-value** (傳值)
 - A **copy** of the argument's **value** (複製參數值) is made and is passed to the called function
 - 註：傳遞複製的參數值給函式 //不影響原變數
 - ▶ In JavaScript, **numbers** (數字), **boolean values** (布林值) and **strings** (字串) are passed to functions by value.
 - ▶ 註：在Javascript中，數字、布林值及字串型態都只是傳值 (pass-by-value).
- ▶ **(2) Pass-by-reference** (傳參考)
 - **The caller allows the called function** to *modify* the data
 - 註：呼叫者允許函數修改傳遞的變數之內容。
 - 註：提高存取效率，避免大量複製資訊，缺點：安全性較低
 - All objects are passed to functions by reference
 - 註：在JavaScript中，所有object傳遞都是傳參考(pass-by-reference).



10.7 Passing Arrays to Functions

function_name (arrayname); //傳遞陣列至函式

- ▶ Pass an array as an argument to a function
 - Specify the array's name without brackets
 - 註：傳遞陣列給函式時，使用陣列名稱即可（不用加“中括號”）
- ▶ Although **entire arrays** are **passed by reference** (array算一種物件), *individual numeric and boolean array elements* (單一元素) are **passed by value** exactly as simple numeric and boolean variables are passed
 - 註：傳遞單一的陣列元素，則是傳值 (*passed-by-value*)
 - 註：這類單一的資料，稱為：**scalars or scalar quantities** (純量)
 - 註：使用方式：*arrayname[元素編號]*，ex: *x[3]*
- ▶ *arrayname.join(“...”);* //當陣列轉字串時，自動在元素之間插入“特殊間隔”
 - Returns a **string** that contains all of the elements of an array, **separated by the string** supplied in the function's argument
 - 註：將陣列轉成字串，並在陣列元素間插入“指定間隔”，ex: 插入“ ” (空白)、插入“,” (逗號)
 - 註：若未指定間隔，則會插入“空字串”(empty string)，ex: 12345678910 (連續)，則不會間斷

10.7 Passing Arrays to Functions (Cont.)

Example:

- 比較：傳陣列元素、傳陣列物件

```
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.13: PassArray.html -->
4 <!-- HTML document that demonstrates passing arrays and -->
5 <!-- individual array elements to functions. -->
6
7 <html>
8   <head>
9     <meta charset = "utf-8">
10    <title>Arrays as Arguments</title>
11    <link rel = "stylesheet" type = "text/css" href = "style.css">
12    <script src = "PassArray.js"></script>
13  </head>
14  <body>
15    <h2>Effects of passing entire array by reference</h2>
16    <p id = "originalArray"></p>
17    <p id = "modifiedArray"></p>
18    <h2>Effects of passing array element by value</h2>
19    <p id = "originalElement"></p>
20    <p id = "inModifyElement"></p>
21    <p id = "modifiedElement"></p>
22  </body>
23 </html>
```

(JavaScript來源)

(2個空id · 輸出用)

(3個id · 輸出用)

```
1 // Fig. 10.14: PassArray.js
2 // Passing arrays and individual array elements to functions.
3
4 function start()
5 {
6   var a = [ 1, 2, 3, 4, 5 ];
7
8   // passing entire array
9   outputArray( "Original array: ", a,
10    document.getElementById( "originalArray" ) );
11   modifyArray( a ) // array a passed by reference
12   outputArray( "Modified array: ", a,
13    document.getElementById( "modifiedArray" ) );
14
15   // passing individual array element
16   document.getElementById( "originalElement" ).innerHTML =
17     "a[3] before modifyElement: " + a[ 3 ];
18   modifyElement( a[ 3 ] ); // array element a[3] passed by value
19   document.getElementById( "modifiedElement" ).innerHTML =
20     "a[3] after modifyElement: " + a[ 3 ];
21 } // end function start()
22
23 // outputs heading followed by the contents of "theArray"
24 function outputArray( heading, theArray, output )
25 {
26   output.innerHTML = heading + theArray.join( " " );
27 } // end function outputArray
28
29 // function that modifies the elements of an array
30 function modifyArray( theArray )
31 {
32   for ( var j in theArray )
33   {
34     theArray[ j ] *= 2;
35   } // end for
36 } // end function modifyArray
37
38 // function that modifies the value passed
39 function modifyElement( e )
40 {
41   e *= 2; // scales element e only for the duration of the function
42   document.getElementById( "inModifyElement" ).innerHTML =
43     "Value in modifyElement: " + e;
44 } // end function modifyElement
45
46 window.addEventListener( "load", start, false );
```

(陣列宣告)

(陣列輸出)

(陣列物件傳遞
修改: pass-by-ref)

(陣列"元素"傳遞
修改: pass-by-value)

(3) 輸出陣列

(文字說明)(陣列變數)(輸出id物件)

(陣列轉字串 · 元素間插入" "空格)

(4) 修改陣列

(for... in 迴圈; 取得Array個數)

(元素值*2)

(5) 修改陣列元素

(1) 註冊event handler: 當load時 · 執行start函式

10.4.3 Summing the Elements of an Array with for and for...in



(輸出)



Effects of passing entire array by reference

Original array: 1 2 3 4 5

Modified array: 2 4 6 8 10

Effects of passing array element by value

a[3] before modifyElement: 8

Value in modifyElement: 16

a[3] after modifyElement: 8

(傳遞陣列物件: pass-by-reference · 內容值會被修改)

(傳遞“單一”陣列元素: pass-by-value · 陣列元素值不會被修改)



10.9 Searching Arrays with Array Method `indexOf`

Searching Arrays 陣列搜尋

- ▶ To determine **whether** an array contains **a value** that matches a certain **key value** (指定數值).
 - 註：判斷此陣列是否包含指定數值
- ▶ The process of locating a particular element value in an array is called *searching*.
 - 註：“陣列搜尋”指的找出陣列中某個元素的“位置”。

`arrayname.indexOf( 搜尋值 );` //由“前→後”搜尋指定值 (從陣列索引0開始搜，並回傳 **index**)

`arrayname.lastIndexOf( 搜尋值 );` //由“後→前”搜尋指定值，並回傳找到的**index**

- ▶ The **built-in** (內建) methods **indexOf** and **lastIndexOf** for searching arrays.
 - Method **indexOf** searches for the **first occurrence** (由前往後找) of the specified **key value**.
 - 註：由“前往後”搜尋
 - Method **lastIndexOf** searches for the **last occurrence** (由後往前找) of the **specified key value**.
 - 註：由“後往前”搜尋
- ▶ If the **key value** is found in the array, each method **returns the index** (回傳索引值) of that value; otherwise, **-1** is returned.
 - 註：**key value** 搜尋到時，回傳該元素索引值，否則回傳**-1** (表示找不到)

10.9 Searching Arrays with Array Method `indexOf` (Cont.)

`inputelement.value` //取得某物件的內容值

- ▶ Every **input element** has a **value** property that can be used to **get** (讀取) or **set** (寫入) the element's value.
 - 註：取得網頁 element的value內容 (ex: `textbox.value = 5`)

(例如：從第5個開始搜)

*`arrayname.indexOf(搜尋值, 起始位置);` //指定“起始位置”開始搜尋
`arrayname.lastIndexOf(搜尋值, 起始位置);`*

- ▶ An **optional second argument** to methods **indexOf** and **lastIndexOf** that represents the **index** from **which to start** (從何處開始搜尋) the search.
 - 註：第二欄參數是指定搜尋起始位置
 - 註：預設不填，啟始位置為從位置0開始
 - 註：若填入的啟始位置超出陣列長度，則回傳-1
 - 註：若填入的啟始位置為負值，則為倒數 (ex: `-1` 代表“倒數第一個位置”)

10.9 Searching Arrays with Array Method `indexOf` (Cont.)

Example:

搜尋陣列之索引值

```
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.17: search.html -->
4 <!-- HTML5 document for searching an array with indexOf. -->
5 <html>
6 <head>
7 <meta charset = "utf-8">
8 <title>Search an Array</title>
9 <script src = "search.js"></script>
10 </head>
11 <body>
12 <form action = "#">
13 <p><label>Enter integer search key:
14 <input id = "inputVal" type = "number"></label>
15 <input id = "searchButton" type = "button" value = "Search">
16 </p>
17 <p id = "result"></p>
18 </form>
19 </body>
20 </html>
```

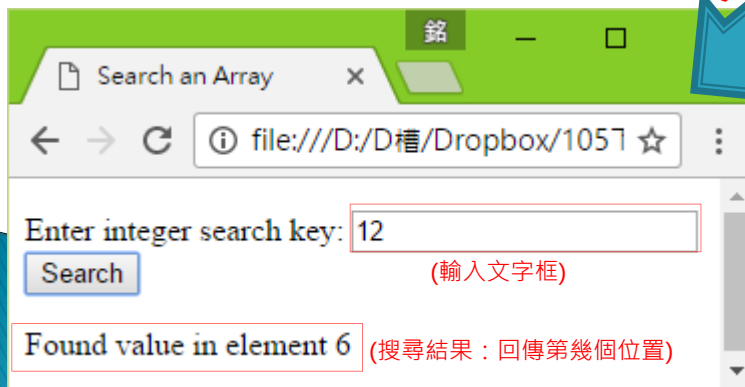
(JavaScript來源)

(2個輸入型id: 數字框、按鈕)

(空id, 輸出用)

(4) `buttonPress` 函式

(輸出)



(輸入文字框)

(搜尋結果: 回傳第幾個位置)

```
1 // Fig. 10.18: search.js
2 // Search an array with indexOf.
3 var a = new Array( 100 ); // create an array
4
5 // fill array with even integer values from 0 to 198
6 for ( var i = 0; i < a.length; ++i )
7 {
8     a[ i ] = 2 * i; // 寫入偶數數值: (0 ~ 198)
9 } // end for
10
11 // function called when "Search" button is pressed
12 function buttonPressed()
13 {
14     // get the input text field
15     var inputVal = document.getElementById( "inputVal" );
16
17     // get the result paragraph
18     var result = document.getElementById( "result" );
19
20     // get the search key from the input text field the perform the search
21     var searchKey = parseInt( inputVal.value ); // (取得id輸入值)
22     var element = a.indexOf( searchKey ); // (往後搜尋)
23
24     if ( element != -1 ) // 找到: 印出該編號 (否則為-1)
25     {
26         result.innerHTML = "Found value in element " + element;
27     } // end if
28     else // 找不到
29     {
30         result.innerHTML = "Value not found";
31     } // end else
32 } // end function buttonPressed
33
34 // register searchButton's click event handler
35 function start()
36 {
37     var searchButton = document.getElementById( "searchButton" );
38     searchButton.addEventListener( "click", buttonPressed, false );
39 } // end function start
40
41 window.addEventListener( "load", start, false );
```

(2) start 函式: (3) 註冊 event handler: 當click時, 執行buttonPressed函式

(1) 註冊 event handler: 當load時, 執行start函式

(找到的索引值)

10.10 Multi-dimensional Arrays

2維陣列

- ▶ To identify a **two-dimensional (2維)** or **multidimensional array (多維陣列)** element
 - By specifying the **two indices (2層array)**
 - 註：透過**2層array**方式宣告
 - By convention, the **first identifies (第一層)** the **element's row**, and the **second identifies (第二層)** the **element's column**
 - 註：第一層是宣告陣列的“**列元素**”，第二層是宣告陣列的“**行元素**”

m-by-n array (m x n 維陣列)

- ▶ An array with **m rows (列)** and **n columns (行)** is called an **m-by-n array (m x n 維陣列)**

arrayname[列索引][行索引]; // 2維陣列元素的存取

- ▶ **Two-dimensional array element accessed using an element name of the form `arrayname[ row ][ column ]`**

- 註：列索引及行索引是用來識別指定元素

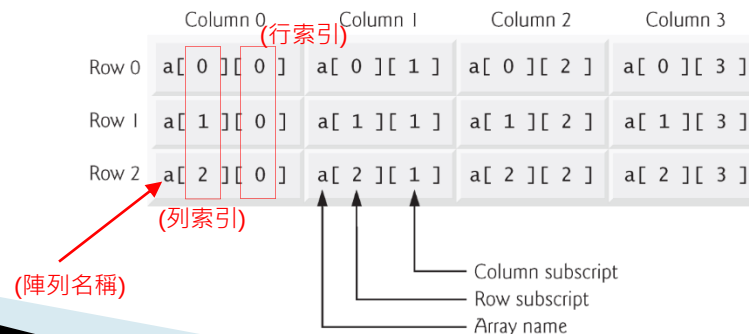


Fig. 10.19 Two-dimensional array with three rows and four



10.10 Multidimensional Arrays (Cont.)

// 在每一欄中，再插入一陣列

arrayname[[...], [...], ...]; // 多維陣列宣告

- ▶ **Multidimensional arrays** can be **initialized** (初始化) in declarations **like** a **one-dimensional array** (如同一維陣列宣告), with **values grouped by row** (再插入子陣列) in square brackets
 - 註：Javascript 多維陣列宣告，如同一陣列宣告，在列的欄位內再插入子陣列。
 - 註：瀏覽器會去每一列計算有多少行元素。
- ▶ A **multidimensional array** (多維陣列) in which each row has a **different number of columns** (不同行元素個數) can be allocated dynamically with operator **new**
 - ▶ 註：多維陣列的行元素個數可以不同。
 - ▶ 如：

```
var array1 = [ [ 1, 2, 3 ], // row 0
               [ 4, 5, 6 ] ]; // row 1
var array2 = [ [ 1, 2 ], // row 0
               [ 3 ], // row 1
               [ 4, 5, 6 ] ]; // row 2
```

(2列3行)

(特別: 行元素個數不同)

(3列3行)

10.10 Multidimensional Arrays (Cont.)

Example: 二維陣列

```
1 <!DOCTYPE html>
2
3 <!-- Fig. 10.13: InitArray3.html -->
4 <!-- HTML5 document showing multidimensional array initialization. -->
5
6 <html>
7   <head>
8     <meta charset = "utf-8">
9     <title>Multidimensional Arrays</title>
10    <link rel = "stylesheet" type = "text/css" href = "style.css">
11    <script src = "InitArray3.js"></script>
12  </head>
13  <body>
14    <h2>Values in array1 by row</h2>
15    <div id = "output1"></div>
16    <h2>Values in array2 by row</h2>
17    <div id = "output2"></div>
18  </body>
19 </html>
```

(JavaScript來源)

(2個空id · 輸出用)

```
1 // Fig. 10.13: InitArray3.js
2 // Initializing multidimensional arrays.
3 function start()
4 {
5   // (多維陣列宣告)
6   var array1 = [ [ 1, 2, 3 ], // row 0
7                 [ 4, 5, 6 ] ]; // row 1
8   var array2 = [ [ 1, 2 ], // row 0
9                 [ 3 ], // row 1
10                 [ 4, 5, 6 ] ]; // row 2
11
12   outputArray( "Values in array1 by row", array1,
13               document.getElementById( "output1" ) );
14   outputArray( "Values in array2 by row", array2,
15               document.getElementById( "output2" ) );
16 } // end function start
17
18 // display array contents
19 function outputArray( heading, theArray, output )
20 {
21   var results = "";
22
23   // iterates through the set of one-dimensional arrays (for..in迴圈)
24   for ( var row in theArray ) //每一列..
25   {
26     results += "<ol>";
27
28     // iterates through the elements of each one-dimensional array (for..in迴圈)
29     for ( var column in theArray[ row ] ) //每一行中..
30     {
31       results += "<li>" + theArray[ row ][ column ] + "</li>";
32     } // end inner for
33
34     results += "</ol>"; // end ordered list
35   } // end outer for
36
37   output.innerHTML = results;
38 } // end function outputArray
39
40 window.addEventListener( "load", start, false );
```

(2) start函式

(5) 輸出函式

(1) 註冊event handler: 當load時 · 執行start函式

Multidimensional Ar X

file:///D:/D槽/Dropbox/1051

Values in array1 by row

1	2	3
4	5	6

(行元素個數相同的二維陣列)

Values in array2 by row

1	2	
3		
4	5	6

(行元素個數不同的二維陣列)



Lab

(上機練習)

Question #1

- ▶ 思考: What is the **output** (輸出結果) if we change **Line #5** in previous example (p.13) to the following code?

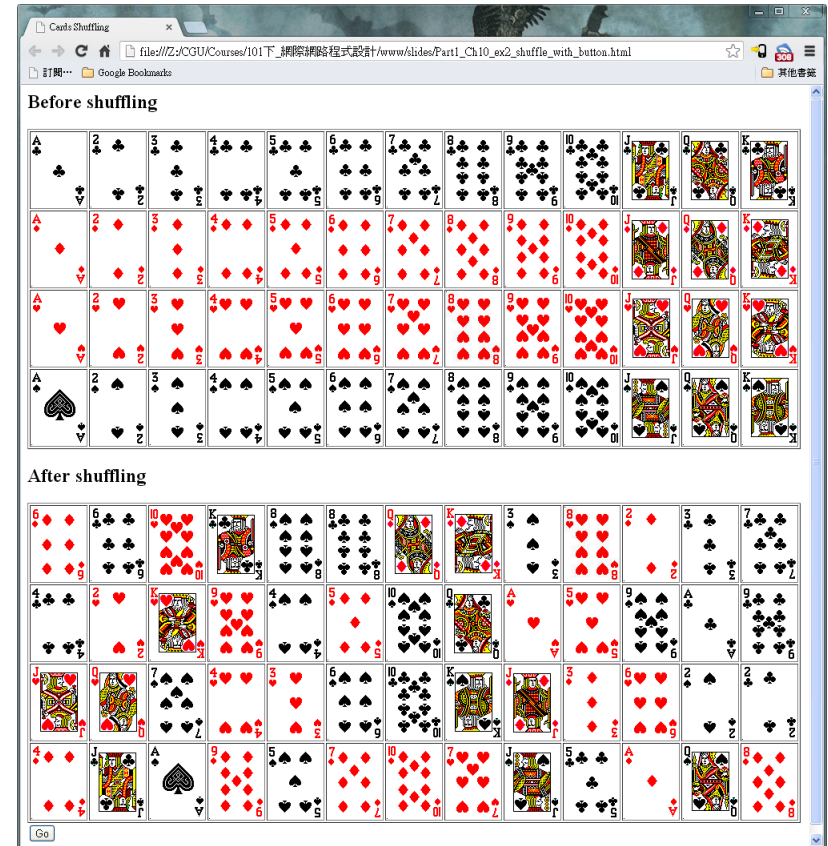
- `var theArray = [ 1, 2, , , 5, 6, 7, 8, 9, 10 ];`

(若未定義)

試試看!

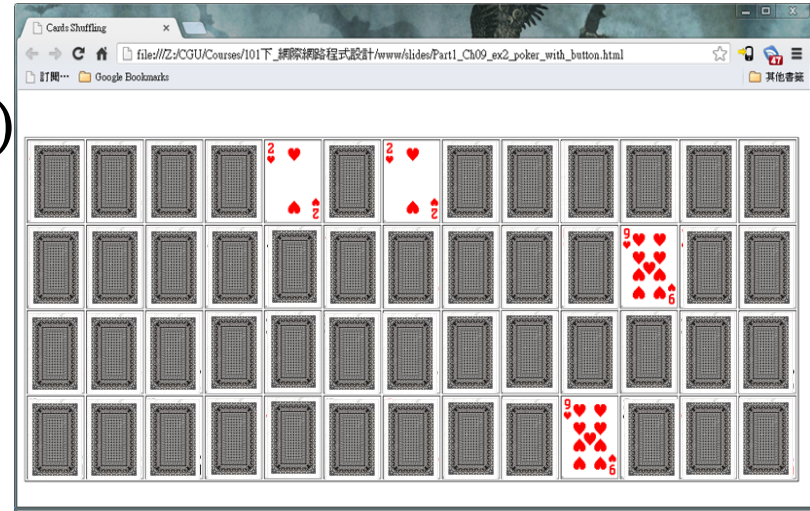
In-Class Exercise #1

- ▶ Modify the JavaScript program you wrote in Ch. 9 to **shuffle (洗牌)** a deck of cards and produce the following tables **by array**.
- ▶ 課中練習：
 - 請嘗試修改Lab5-2，達成洗牌動作
 - 注意：花色數字**不允許重覆**
 - 使用陣列方式



(Bonus) In-Class Exercise #2

- ▶ 小遊戲 (對對樂) :
 - 1. 牌中只含紅心1~13 (偶數張)
 - 2. 隨機散佈在52格中(蓋牌)
 - 3. 使用者每次可翻開2張牌
 - 若數字不相同, 則蓋回
 - 若數字相同, 則不蓋回
 - 4. 若所有牌子成功翻開, 顯示文字: 恭喜獲得勝利~!



- 注意:

本週五(5/22) 23:59 前上傳到ftp (否則視為遲交):

<ftp://111webdesign:webdesign@120.114.190.22:4000>

Lab08

- 請以"學號"建立資料匣 (否則扣分)

★提醒: 記得上傳期中報告 (FTP/上傳區/期中分組報告)



公告：期末專題Demo報告

- ▶ 預計報告日期：
 - 第16週 (6/8) (ps. 如想延後至第17週Demo，可以來信，老師會舉辦投票)
- ▶ 報告規則：
 - 每組報告時間：15分鐘 (最多20分鐘，含Q&A)
 - 每組報告順序，依先前分組報告順序，公佈於：
 - https://docs.google.com/spreadsheets/d/1lCve_2rLigQKjC85pGJUttPQ3w8PAHBbBA2GGFTQrJA/edit?usp=sharing
- ▶ Demo報告評分標準：
 1. 專題創意性(25%)
 2. 技術困難性(25%)
 3. 實作完整性(25%)
 4. 展示設計/報告流暢性(25%)
- ▶ PS. 額外加分
 - 報名校外競賽
 - 證明方式：放入簡報 & FTP上傳報名資料及報名成功之Email截圖紀錄，包含：競賽名稱，作品名稱，小組成員，指導老師 (梁，亦可加其他老師))
 - 報名成功：期末專題加分(+2.5分)
 - 獲得名次：再加分(+2.5~7.5分：佳作、第三名、第二名、第一名)



【工商服務】



分享：老師指導專題生-近期成果

- 【2023/5/12】
 - ▶ 112年度 U-start 創新創業計畫
 - 獲得：第一階段獲補助團隊
 - 獎金：50萬元
 - 新聞連結：
 - <https://money.udn.com/money/story/5723/7165374>



準公司、準商業及準有限合夥設立一站式線上申請專屬使用者憑證密碼函	
憑證卡號	1002000003337422
憑證序號	A095E9273E96F6A22A616EA1D065619B
發行者名稱	C=TW,O=行政院,OU=工商憑證管理中心
持有者資訊	CN=準公司、準商業及準有限合夥設立一站式線上申請專屬使用者,SerialNumber=112034133,O=智輪科技有限公司,C=TW
預查編號	112034133
識別資料	智輪科技有限公司

南大首頁



國立臺南大學
National University of Tainan

網站導覽 | 回首頁 | English |

認識南大

在校學生

教職員工

校友

未來學生

訪客

南大入口 (MYNUTN)

招生與入學

太優秀!!

南大培育創業菁英 U-start創新創業計畫 獲150萬元獎金



教學單位

行政單位

校務資訊系統

招生與入學

校務暨財務資訊

意見交流

International Students

南大防疫活動異動通知

南大高教深耕雲端學院防疫懶人包

全文檢索 (全文檢索)

GO

字級設定: 小 中 大

榮譽榜

南大報導



~送給同學們~❤

人類因**夢想**而偉大

Davis Yang